

Agentic Coding Build Session

Four collaborative project ideas for Virginia Tech Computer Science graduate students

10-15 participants | Five teams of two or three | 60-90 minutes

The series has introduced prompt engineering, Skills, and the basics of Claude Code and Codex, including /plan and /goal. This session moves from instruction to building something together.

One shared outcome, with clear ownership

Aim for one shared outcome built by five teams of two or three people. That gives everyone something to own without putting 15 people into the same file. Teams can work independently on their assigned components while discussing progress and testing one another's work.

Separate using agents to build software from building software that requires an AI service. For this session, avoid new API integrations, accounts, databases, or deployment requirements. Coding assistants can do substantial work without those becoming part of the finished application.

The four project options

Project	How people collaborate	Final demonstration	Best fit
1. Grad Student Toolbox	Each team contributes a small tool to one shared website.	A working collection of five useful tools.	Safest first collaborative build
2. Bot Battle Arena	Teams write competing strategies for the same tournament runner.	A live tournament with a scoreboard.	Most playful and social
3. Paper Connections Explorer	Teams build parts of a research-paper browsing application.	Paper cards, related papers, and a reading shortlist.	Most relevant to graduate research
4. Meeting-to-Action Pipeline	Teams build stages that transform the same input into useful outputs.	Notes become validated tasks, a board, and a Markdown summary.	Strongest continuation of the Skills lessons

These are workshop-sized proposals, not production applications. Target the first two for a 60-minute session and give the latter two the full 90 minutes.

Recommendation for today: Choose the Grad Student Toolbox for a first shared build. Choose Bot Battle Arena when the priority is a playful session with a lively final demonstration.

PROJECT 01 / TARGET: 60 MINUTES

Grad Student Toolbox

"By the end of tonight, our group will have built a website that contains five tools we might actually use."

Each team creates one small browser-based utility. Combine them through a shared landing page called something like Hokie Grad Toolbox.

Team assignments and minimum working behavior

Team	Tool	Minimum working behavior
1	Focus timer	Start, pause, and reset a work/break timer.
2	Deadline tracker	Add a task and deadline, then display the remaining time.
3	Writing-length checker	Paste text, count words, and show progress against a target.
4	JSON workbench	Paste JSON, format valid input, and clearly report invalid input.
5	Decision matrix	Enter alternatives, criteria, and weights, then calculate a ranking.

How the pieces connect

Give every team a separate folder, such as `tools/focus-timer/`, containing its own page and supporting files. The facilitator or a designated integration lead owns the landing page. For the first version, that page simply links to each tool. A sophisticated plugin system is not necessary.

Use plain HTML, CSS, and JavaScript unless everyone already shares another working setup. Tools that need persistence can use `localStorage`, which retains data across browser sessions. Make it clear that this is local to that browser and site, not a shared database for the group. [1]

What students practice

Writing a small specification, constraining an agent's scope, testing behavior, and delivering something another person can use. Have each team exchange its tool with another team for a short usability test.

Finish line and scope

Definition of done: All five tools open from the landing page, each performs its core function, and another team has successfully tested it.

Keep out of scope: Authentication, shared user accounts, cloud synchronization, and a common backend. Styling consistency is a bonus, not a prerequisite.

Why this fits today: Everyone can make progress independently, but the final result still belongs to the group. An unfinished tool does not prevent the other four from working.

Documentation [1]: [MDN Web Docs: Window.localStorage](https://developer.mozilla.org/en-US/docs/Web/API/Window/localStorage)

PROJECT 02 / TARGET: 60 MINUTES

Bot Battle Arena

"Use your coding agent to build a strategy, then compete against everyone else's code."

Use repeated rock-paper-scissors rather than build a complicated game engine. The interesting problem is not choosing a move once. It is deciding whether an opponent has a pattern and adapting over many rounds.

One shared interface

Each bot implements the same small interface:

```
choose_move(my_history, opponent_history)
-> "rock" | "paper" | "scissors"
```

Both histories contain only completed rounds. Neither bot sees its opponent's current choice before submitting its own.

How people collaborate

Four teams build bots. The fifth team owns the tournament runner, input validation, tests, and scoreboard. Provide a baseline random bot and a few predictable practice opponents so strategy teams can start without waiting for the tournament infrastructure.

Teams can explore different strategies: recognizing repeating sequences, tracking an opponent's recent preferences, combining several predictors, or deciding when there is insufficient evidence to do anything except choose randomly.

What actually gets built

A Python runner loads the bots, plays repeated matches, and prints a standings table. Python's standard library includes unittest, so the group can write and run tests without adding a separate testing framework. [2]

What students practice

Specifying an interface, generating and testing algorithms, inspecting failures, and improving a solution against measurable behavior. Ask each team to make one prediction before testing: "Our bot should do well against this kind of opponent because..."

Finish line, scope, and preparation

Definition of done: Every bot completes a match without crashing, the runner handles invalid moves consistently, and the group can run a tournament. Treat the result as a game, not evidence that one coding model is generally better than another.

Keep out of scope: Live LLM calls for individual moves, networking, elaborate animation, and reinforcement-learning training.

Preparation that matters: Have a working minimal runner ready beforehand. The infrastructure team can improve it rather than becoming a dependency that everyone waits on. A visual scoreboard is a 90-minute stretch goal.

Why this fits the group: People can code separately while still having a strong reason to discuss strategies, compare results, and cheer during the final demonstration.

Documentation [2]: [Python documentation: unittest](#)

PROJECT 03 / TARGET: 90 MINUTES

Paper Connections Explorer

"Build a small application that helps us discover connections between papers we are reading."

Start with a prepared collection of approximately 10-15 paper records. Each record contains an identifier, title, year, abstract, and a few manually assigned tags. Use public metadata or clearly labeled synthetic examples rather than making PDF extraction part of the exercise.

The application lets someone browse paper cards, filter by topic, select a paper, and see other papers with overlapping tags.

How people collaborate

Split the work into five parts: a data loader and validator; the application shell with search and paper cards; related-paper scoring; a simple visual connection view; and a reading shortlist with export.

Agree on the record format before anyone starts coding. JSON Schema provides a standard way to specify required properties and their types, making it suitable for checking that each team's records follow the same contract. [3]

Keep the recommendation logic deliberately simple

Rank related papers by shared tags. Label the result "shared topics," not "semantic similarity" or "citation relationships." The application should show what its calculation actually means.

For the visual view, start with a selected paper in the center and related papers around it. Do not make an elaborate force-directed graph necessary for the core application to function.

What students practice

Building against a shared data format, separating data processing from presentation, coordinating components, and making computed results understandable to users.

Finish line, scope, and preparation

Definition of done: A person can load the sample collection, filter the cards, select a paper, inspect its shared topics with another paper, and save a reading shortlist.

Keep out of scope: PDF parsing, embeddings, vector databases, automatic citation discovery, and live scholarly-search integrations.

Preparation that matters: Give every team the same sample data and a minimal running application. Native JavaScript modules can separate the components without requiring a large framework, but serve the application through a local web server rather than relying on opening module-based pages directly from disk. [4]

Why this fits the group: It produces a research-oriented application with visible contributions from multiple teams. It is also the most integration-heavy option, so reserve it for the full 90 minutes.

Documentation [3]: [JSON Schema: Getting started step-by-step](#)

Documentation [4]: [MDN Web Docs: JavaScript modules](#)

Meeting-to-Action Pipeline

“Build a workflow that turns messy meeting notes into something a team can act on.”

Use a short, fictional meeting transcript. The finished workflow extracts proposed tasks and decisions, validates the structured output, and generates a task board and a Markdown summary. This is the most direct way to turn the previous Skills instruction into a shared coding project.

Team assignments and deliverables

Team	Component	Deliverable
1	Extraction skill	Instructions and tests for producing the agreed JSON structure from notes.
2	Validator	Code that checks required fields, valid values, and duplicate identifiers.
3	Task board	A browser view that loads the JSON and displays tasks by status.
4	Summary generator	Code that converts structured data into a Markdown meeting summary.
5	Pipeline and testing	A runner that connects the coded stages and tests the complete workflow.

A Claude Code skill can package a SKILL.md instruction file together with supporting scripts and examples, so the extraction step can build directly on what the group has already learned. [5]

The critical collaboration decision

Supply one example JSON file before coding begins. Everyone builds against that file immediately. The board and summary teams should not have to wait for the extraction team.

Keep the extraction rules explicit: do not invent owners or deadlines; use null when information is missing; preserve a short source excerpt for each extracted task. Validation should check the structure, while a human compares the extracted content with the original notes.

What students practice

Reusable Skills, structured output, component boundaries, automated validation, and distinguishing “the agent produced something” from “the result passed our checks.”

Finish line and scope

Definition of done: Invoke the extraction skill through an existing coding assistant, save its output, run the validator, and open the resulting board and summary. A prepared fixture should also let the coded pipeline run independently of the extraction step.

Keep out of scope: Recording audio, speech recognition, autonomous multi-agent orchestration, and sending anything to real email or calendar accounts.

Why this fits the group: The teams genuinely depend on one another's outputs, but a shared fixture lets them work in parallel. The result demonstrates how Skills and conventional code can work together.

Documentation [5]: [Claude Code documentation: Skills](#)

How to Run the Session

Pick one project for the whole room rather than assign all four projects simultaneously. Use the schedule below to protect time for integration, cross-testing, and a shared demonstration.

60-minute and 90-minute schedules

Phase	60-minute session	90-minute session	Activity
Define the finish line	5 minutes	5 minutes	Show the intended demo and name what is out of scope.
Form teams and agree on interfaces	5 minutes	10 minutes	Assign folders, responsibilities, and sample inputs/outputs.
Build	25 minutes	40 minutes	Each team plans, implements, and tests its component.
Integrate and cross-test	15 minutes	20 minutes	Connect the pieces and have another team try them.
Demonstrate and reflect	10 minutes	15 minutes	Run the shared result and discuss what required human judgment.

Roles that keep everyone involved

For pairs, use a driver and reviewer. For trios, add a tester. Rotate halfway through so the session does not become one person operating an agent while everyone else watches.

Shared starting instruction for every team

We are building one component of a shared workshop project. Read the project specification and sample inputs first. Propose a small implementation plan and three acceptance tests before editing. Work only in our assigned folder. Do not change shared interfaces or add dependencies without discussing it with the integration lead. Build the smallest working version first, then run the tests and report what works, what failed, and what remains untested.

Recommended choice for this session

Start with the Grad Student Toolbox. It gives the group a realistic first experience of “we built something together” without making integration the entire evening. Choose Bot Battle Arena instead when the priority is a playful session with a lively final demonstration.

The Paper Connections Explorer and Meeting-to-Action Pipeline are the fuller 90-minute options: the former centers on research workflows and interface integration, while the latter extends the group's Skills lessons into a shared, validated workflow.

Source note: Documentation links from the project discussion are retained in the relevant project briefs. Project scope, team assignments, and timing are workshop design proposals.